

Guía de inicio: Análisis automatizado de pliegos con BMAD Method sobre Azure

Qué descargar, cómo instalar y usar BMAD Method, cómo funciona el ecosistema de Azure (Document Intelligence, AI Search, OpenAI) y cómo arrancar el desarrollo del pipeline de extracción de características de licitaciones.

METODOLOGÍA

BMAD Method

STACK

Python · LangGraph · Azure (Document Intelligence, AI Search, OpenAI)

DESPLIEGUE

Azure Container Apps

1 Resumen del proyecto

El objetivo es extraer de forma automatizada y estructurada, de documentos tipo pliego, las siguientes características: **objeto y alcance, requisitos de admisibilidad, garantías, plazos clave, criterios de evaluación, causales de rechazo y anexos obligatorios**. La salida se guarda como **JSON estructurado** (para consultas exactas) y también **vectorizada** (para búsqueda semántica).

El desarrollo se organiza con **BMAD Method**, framework de planificación ágil asistida por agentes de IA (brief, PRD, arquitectura, historias) que después se implementan una por una.

DECISIÓN DE ARQUITECTURA VIGENTE

Stack **100% Azure desde el día uno**: Azure AI Document Intelligence (ingesta), Azure AI Search (vectores + búsqueda + metadata), Azure OpenAI (chat y embeddings), LangGraph (orquestración — Microsoft Agent Framework queda descartado por ahora), y despliegue en Azure Container Apps. El código se organiza en capas simples con un adapter liviano por servicio de Azure (ver sección 12) — sin puertos formales, porque no hay migración de nube pendiente.

2 Stack tecnológico

Componente	Herramienta
Lenguaje / runtime	Python 3.11+
Ingesta de documentos	Azure AI Document Intelligence (modelo <code>prebuilt-layout</code>)
Vector store + búsqueda + metadata	Azure AI Search (índice único: texto, vector y campos filtrables)
Modelos de IA (chat + embeddings)	Azure OpenAI
Orquestación de agentes	LangGraph
Validación de datos	Pydantic
Planificación ágil	BMAD Method
Despliegue	Azure Container Apps

3 Qué descargar / checklist de instalación

Requisitos base

- Git
- Node.js LTS (requerido por BMAD Method)
- Python 3.11+
- `uv` (gestor de paquetes y entornos virtuales de Python)
- Azure CLI (`az`)
- GitHub Copilot Chat en VS Code — el editor de este equipo

Recursos de Azure a crear

- Cuenta de Azure (suscripción activa)
- Recurso de **Azure AI Document Intelligence**
- Recurso de **Azure AI Search** (tier Basic o superior para vectores en producción; el tier gratuito alcanza para probar)
- Recurso de **Azure OpenAI**, con un modelo de chat y un modelo de embeddings desplegados
- Azure Container Apps (para el despliegue, más adelante)

```
# Login y verificación de Azure CLI
az login
az account show

# Gestor de paquetes uv (si no lo tenés instalado)

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"

# Mac / Linux
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Entorno virtual del proyecto

```
# Crear y activar el entorno virtual con uv
uv venv
source .venv/bin/activate          # en Windows: .venv\Scripts\activate

# Librerías del pipeline
uv pip install langgraph langchain pydantic
uv pip install azure-ai-documentintelligence
uv pip install azure-search-documents
uv pip install openai azure-identity
```

IMPORTANTE

Agregar `.venv/` al `.gitignore` antes del primer commit. BMAD Method corre con Node/npx y no usa este entorno virtual — solo lo necesitan las librerías del pipeline en sí.

4 BMAD Method — guía completa paso a paso

Qué es: framework open source de planificación ágil asistida por agentes de IA. Provee agentes especializados (analista, PM, arquitecto, dev) que guían la creación de brief, PRD, arquitectura e historias mediante *workflows* — no comandos de barra, sino *skills* que se invocan por nombre en el IDE de IA.

Cómo funciona esto en la práctica

No es un chat aparte con "un agente" — es la misma ventana de **GitHub Copilot Chat** donde ya se escribe normalmente. Instalar BMAD le agrega a esa herramienta un conjunto de "recetas" (skills) que reconoce por nombre. Escribir el nombre de una skill es como mandar un mensaje cualquiera, solo que en vez de responder libre, la IA sigue un guión: hace preguntas una por una en esa misma conversación y, al final, **escribe sola los archivos** en el proyecto (nadie los edita a mano).

PRIMERA SESIÓN, PASO A PASO

1. Con GitHub Copilot Chat abierto en la carpeta del proyecto (en VS Code), escribir `bmad-help` — literal, como un mensaje más.
2. Contarle de qué se trata con las propias palabras: "Es un sistema que analiza pliegos de licitación en PDF y extrae objeto, requisitos, garantías, plazos, criterios de evaluación, causales de rechazo y anexos. Ya tenemos un frontend armado y decidimos el stack técnico."
3. `bmad-help` recomienda el próximo paso — probablemente `bmad-product-brief`.
4. Se corre ese comando y arranca una charla de ida y vuelta: la IA pregunta (quién lo usa, qué problema resuelve, alcance del MVP), uno responde en lenguaje natural.
5. Al terminar, aparece solo el archivo (ej. `_bmad-output/planning-artifacts/product-brief.md`), ya redactado.
6. Para el siguiente workflow (ej. `bmad-prd`) se abre un **chat nuevo** — no se sigue en la misma conversación — y se repite la misma dinámica.

Dos tipos de skills en Copilot: agentes vs workflows

En GitHub Copilot (VS Code) hay un menú desplegable de "Agent" que muestra opciones como `bmad-agent-dev`, `bmad-agent-analyst`, `bmad-agent-pm`, etc. Es fácil confundirlo con la lista completa de skills, pero solo muestra **un tipo** de las dos que existen:

Tipo	Qué hace	¿Aparece en el menú "Agent" de Copilot?
Skills de agente (<code>bmad-agent-dev</code> , <code>bmad-agent-analyst</code> ...)	Cargan una persona completa con su propio menú de opciones — el agente queda "en personaje" toda la charla	Sí
Skills de workflow (<code>bmad-prd</code> , <code>bmad-architecture</code> , <code>bmad-ux</code> , <code>bmad-product-brief</code> , <code>bmad-help</code> ...)	Corren un proceso puntual, sin cargar una persona primero	No — Copilot solo lista ahí las que son personas

CÓMO INVOCAR CADA TIPO

Los **agentes** se seleccionan del menú desplegable de Copilot. Una vez cargado, el agente muestra su propio menú de opciones — ahí se le puede pedir en lenguaje natural (ej. "quiero armar el product brief") o con el atajo con asterisco (ej. `*product-brief`). Los **workflows** no aparecen en ese menú — probar escribiendo `/` en el chat para ver si aparece como slash command (ej. `/bmad-help`), en vez de escribir el nombre a secas.

Qué agente seleccionar para cada parte de este proyecto:

Qué se quiere hacer	Agente a seleccionar
Product brief	<code>bmad-agent-analyst</code>
PRD, épicas e historias	<code>bmad-agent-pm</code>
Diseño del frontend (UX)	<code>bmad-agent-ux-designer</code>
Arquitectura, chequeo de coherencia	<code>bmad-agent-architect</code>
Sprint planning, crear historia, implementar, code review, retro	<code>bmad-agent-dev</code>

Otros agentes que pueden aparecer en el menú (`bmad-agent-tech-writer`, `bmad-agent-builder`, los `bmad-cis-agent-*`, `bmad-tea`) son de módulos que no hacen falta para este proyecto — documentación técnica extra, personalización de BMAD, y agentes de ideación/testing de otros módulos que vinieron con la instalación base.

Prerrequisitos

- **Node.js 20.12+** (lo necesita el instalador)

- **Git** (recomendado, para versionar)
- **GitHub Copilot Chat** en VS Code, con suscripción activa
- Una idea de proyecto — ya la tenemos

Paso 1 — Instalar

```
# Parado en la raíz del proyecto
npx bmad-method install
```

El instalador pregunta, entre otras cosas, con qué módulo instalar (elegir **BMad Method**) y con qué herramienta/IDE integrarlo — ahí elegir **GitHub Copilot** de la lista. Crea dos carpetas:

- `_bmad/` — agentes, workflows y configuración
- `_bmad-output/` — vacía al principio; ahí se van a guardar todos los documentos que generen los agentes

SI NO APARECE "GITHUB COPILOT" EN LA LISTA

Correr `npx bmad-method install --list-tools` antes de instalar, para ver el identificador exacto de todas las herramientas soportadas en la versión que están instalando.

Paso 2 — Primer contacto: `bmad-help`

Con GitHub Copilot Chat abierto en la carpeta del proyecto, invocar la skill:

```
bmad-help
```

Detecta qué módulos están instalados, qué se hizo hasta ahora, y recomienda el próximo paso. También se le puede preguntar directo, por ejemplo: `bmad-help tengo un pipeline de análisis de pliegos con frontend ya armado, por dónde empiezo`.

REGLA QUE HAY QUE RESPETAR SIEMPRE

Cada workflow se corre en un **chat nuevo**. No seguir encadenando workflows en la misma conversación — evita que el contexto se llene y genere errores.

Entender las 4 fases

Fase	Nombre	Qué pasa
1	Analysis <i>(opcional)</i>	Brainstorming, investigación, product brief
2	Planning <i>(obligatoria)</i>	PRD (requisitos) y diseño de UX si aplica
3	Solutioning	Arquitectura, y desglose en épicas e historias
4	Implementation	Construcción historia por historia

Elegir el track de planificación

Track	Para qué sirve	Documentos que genera
Quick Flow	Fixes chicos, features simples (1-15 historias)	Solo tech-spec
BMad Method	Productos, plataformas, features complejas (10-50+ historias)	PRD + Arquitectura + UX
Enterprise	Compliance, sistemas multi-tenant (30+ historias)	PRD + Arquitectura + Seguridad + DevOps

TRACK RECOMENDADO PARA ESTE PROYECTO

BMad Method. No es un fix chico (Quick Flow se queda corto: pipeline de 8 nodos + frontend + integraciones futuras), pero tampoco necesitamos compliance/multi-tenant desde el día uno (Enterprise sería sobre-ingeniería ahora).

Fase 1 — Análisis (opcional)

Como el concepto del proyecto ya está claro (lo definimos en este mismo documento), alcanza con generar el **product brief**:

```
bmad-product-brief
```

Fase 2 — Planificación (obligatoria)

```
# En un chat nuevo  
bmad-prd
```

Va a preguntar la intención: **Create** (arrancar de cero), **Update** o **Validate**. Elegir Create. Genera `prd.md`, `addendum.md` y `.memlog.md` dentro de `_bmad-output/planning-artifacts/`.

Diseño de UX (aplica en este proyecto)

El frontend **todavía no está construido** — las capturas que se usaron como referencia eran solo inspiración de cómo podría verse, no una pantalla ya hecha para conectar. Por eso, después del PRD corresponde correr el workflow de diseño:

```
# En un chat nuevo  
bmad-agent-ux-designer  
bmad-ux
```

Acá se define el diseño real de las 3 pantallas (Dashboard, Análisis de Pliegos, Historial) usando las capturas como referencia de estilo, más la futura pantalla de "Radar de oportunidades". El resultado de este paso alimenta la arquitectura y, después, las historias del frontend van a salir junto con las de los nodos del backend en el mismo `bmad-create-epics-and-stories`.

Fase 3 — Solutioning (arquitectura + historias)

1. **Arquitectura:** invocar el agente Architect (`bmad-agent-architect`) en un chat nuevo, correr `bmad-architecture`. Acá se vuelca la arquitectura de capas simples + adapters (sección 12) y los 8 nodos. Genera `architecture.md`.
2. **Épicas e historias:** invocar el agente PM (`bmad-agent-pm`), correr `bmad-create-epics-and-stories`. Usa el PRD + la arquitectura juntos, así que las historias salen ya informadas técnicamente (acá van a aparecer los 8 nodos como historias independientes).
3. **Chequeo de coherencia** (*muy recomendado*): agente Architect de nuevo, correr `bmad-check-implementation-readiness` — valida que PRD, arquitectura e historias sean consistentes entre sí antes de empezar a programar.

Construcción (Paso 2 de BMAD)

```
# Agente Developer, una sola vez al principio  
bmad-agent-dev  
bmad-sprint-planning
```

Esto crea `sprint-status.yaml`, que trackea todas las épicas e historias. Después, por cada historia (en chats nuevos):

Paso	Skill	Qué hace
1	<code>bmad-create-story</code>	Crea el archivo de la historia a partir de la épica
2	<code>bmad-dev-story</code>	Implementa la historia
3	<code>bmad-code-review</code>	Valida calidad (<i>recomendado</i>)

Al terminar todas las historias de una épica: `bmad-retrospective`.

Resultado final

```
proyecto-pliegos/  
├── _bmad/                                # configuración de BMAD  
├── _bmad-output/  
│   ├── planning-artifacts/  
│   │   ├── PRD.md  
│   │   ├── architecture.md  
│   │   └── epics/                        # épicas e historias  
│   ├── implementation-artifacts/  
│   │   └── sprint-status.yaml  
│   └── project-context.md                # opcional  
└── ...
```

Referencia rápida de comandos

Skill	Agente	Para qué
<code>bmad-help</code> ★	Cualquiera	Guía inteligente — preguntar lo que sea
<code>bmad-product-brief</code>	Analyst	Brief inicial del proyecto
<code>bmad-prd</code>	Cualquiera	Crear, actualizar o validar el PRD
<code>bmad-architecture</code>	Architect	Documento de arquitectura
<code>bmad-create-epics-and-stories</code>	PM	Desglosar el PRD en épicas e historias
<code>bmad-check-implementation-readiness</code>	Architect	Validar coherencia de todo lo planificado
<code>bmad-sprint-planning</code>	Dev	Inicializar el tracking de sprint
<code>bmad-create-story</code>	Dev	Crear el archivo de una historia
<code>bmad-dev-story</code>	Dev	Implementar la historia
<code>bmad-code-review</code>	Dev	Revisar el código implementado

Secuencia completa para este proyecto

Todo lo anterior, en el orden real en que se usa:

#	Paso	Frecuencia
1	bmad-product-brief	Planificación — una sola vez , al principio de todo el proyecto
2	bmad-prd	
3	bmad-ux (diseño del frontend, todavía no construido)	
4	bmad-architecture	
5	bmad-create-epics-and-stories	
6	bmad-check-implementation-readiness	
7	bmad-sprint-planning	Una sola vez, arranca la implementación
8	bmad-create-story → bmad-dev-story → bmad-code-review	Se repite por cada historia (15-30 veces aprox.)
9	bmad-retrospective	Al terminar cada épica

Después del MVP: una nueva ronda no es empezar de cero

Cuando llegue el momento de sumar algo nuevo (el Radar de oportunidades, definir y agregar el scoring, etc.), no se rearma el proyecto desde el brief. `bmad-prd` pregunta la intención al arrancar — ahí está la clave:

Intención	Cuándo usarla
Create	Arrancar de cero — es lo que se usa para el MVP, una sola vez
Update	Agregar alcance nuevo sobre el PRD ya existente — esto es lo que se usa después del MVP
Validate	Chequear que el PRD siga siendo consistente, sin agregar nada nuevo

Qué se rehace de cero y qué se actualiza, cuando llegue esa segunda ronda:

Documento	¿Se rehace de cero?	Por qué
product-brief.md	No	El problema que resuelve el proyecto no cambia — sigue siendo el mismo producto, solo crece en alcance
prd.md	Se actualiza (<code>bmad-prd</code> , intención Update)	Acá se agrega el alcance nuevo
architecture.md	Se actualiza (<code>bmad-architecture</code>)	La base de capas + adapters se mantiene; se documentan los adapters/nodos nuevos que hagan falta
Épicas/historias	Se agregan nuevas (<code>bmad-create-epics-and-stories</code>)	Se suman al <code>sprint-status.yaml</code> existente, no lo reemplazan

CUÁNDO SÍ HARÍA FALTA UN BRIEF NUEVO DE VERDAD

Solo si el proyecto pivotara a algo distinto (dejara de ser "análisis de pliegos" y pasara a ser otra cosa). No es el caso esperado acá — esto es crecimiento incremental del mismo producto, no un cambio de rumbo. La cadencia natural es: entregar el MVP, juntar feedback real de uso, y recién ahí abrir una ronda de `bmad-prd` en modo Update — no antes, para no planificar sobre supuestos que todavía no se validaron con el pipeline funcionando.

El paso 8 se repite una vez por historia — el ciclo es siempre el mismo, tanto para una pantalla de frontend simple como para un nodo de extracción complejo. Ver el ejemplo completo en la sección 6.

Qué modelo de Copilot usar en cada paso

"Auto" en Copilot está optimizado para disponibilidad y evitar rate-limiting, no para elegir el mejor modelo según la tarea. Para los pasos de planificación (se corren una sola vez y definen todo lo demás) conviene salir de Auto y elegir a mano:

Tarea	Modelo	Por qué
<code>bmad-product-brief</code> , <code>bmad-prd</code> , <code>bmad-ux</code> , <code>bmad-architecture</code> , <code>bmad-create-epics-and-stories</code>	Claude Opus 4.6	Corren una sola vez y definen todo lo que sigue — un error acá se arrastra a 15-30 historias
<code>bmad-check-implementation-readiness</code>	Claude Sonnet 4.6	Es verificación de coherencia, no creación desde cero
<code>bmad-create-story</code> (historias delicadas, ej. nodos de extracción)	Claude Opus 4.6	Pensar bien los criterios de aceptación y casos borde de variabilidad de redacción vale el modelo más fuerte
<code>bmad-create-story</code> (historias simples, ej. pantallas de frontend)	Claude Sonnet 4.6	Historia autocontenida y de bajo riesgo, un tier medio alcanza
<code>bmad-dev-story</code> , <code>bmad-code-review</code>	GPT-5.3-Codex	Especializado en código, corre muchas veces (una por historia) — pesa el costo/velocidad
<code>bmad-sprint-planning</code> , <code>bmad-retrospective</code>	Auto	Administrativo/templado, bajo riesgo, aprovecha el descuento de Auto

Regla simple para acordarse: cuanto más se **arrastra** el resultado hacia adelante (brief → PRD → arquitectura → historias), más conviene el modelo más fuerte disponible. Cuanto más se **repite** (una vez por historia) y es más mecánico, más conviene Auto o el modelo de código.

5 Ecosistema de Azure — herramientas y cómo interactúan

Tres servicios de Azure hacen todo el trabajo pesado del pipeline. Antes de ver cómo se conectan entre sí, qué es cada uno:

Azure AI Document Intelligence

Servicio de OCR + comprensión de layout basado en ML. Para este proyecto alcanza con el modelo `prebuilt-layout` — no hace falta entrenar un modelo custom para el MVP. Extrae texto, tablas y estructura de PDFs (nativos o escaneados) con el mismo modelo, y puede devolver el resultado directamente en **Markdown**, conservando la jerarquía de secciones — justo lo que necesita el chunking estructural que definimos en la sección 8.

```
from azure.ai.documentintelligence import DocumentIntelligenceClient
from azure.ai.documentintelligence.models import AnalyzeDocumentRequest, ContentFormat
from azure.core.credentials import AzureKeyCredential

client = DocumentIntelligenceClient(endpoint=endpoint, credential=AzureKeyCredential(key))
with open("pliego.pdf", "rb") as f:
    poller = client.begin_analyze_document(
        "prebuilt-layout",
        body=f,
        output_content_format=ContentFormat.MARKDOWN,
    )
result = poller.result()
markdown = result.content # el pliego completo, ya en Markdown con tablas y estructura
```

LÍMITES A TENER EN CUENTA

Hasta 2.000 páginas por PDF. Imágenes entre 50×50 y 10.000×10.000 píxeles. Los PDFs con contraseña necesitan que se les saque el lock antes de analizarlos. Se cobra por página analizada — por eso el resultado se cachea y nunca se vuelve a analizar el mismo pliego dos veces (más detalle en la sección 10, sobre costos).

Azure AI Search

Motor de búsqueda que combina **vector search** (similitud semántica), **keyword search** (texto exacto) y **filtros sobre campos estructurados** — todo en un mismo índice. Cuando una consulta combina las dos primeras formas de búsqueda se llama *hybrid search*: corren en paralelo y los resultados se combinan con Reciprocal Rank Fusion (RRF).

POR QUÉ REEMPLAZA A DOS PIEZAS DEL DISEÑO ANTERIOR

En el diseño híbrido que teníamos antes, usábamos un vector store (ChromaDB) separado de una base para el JSON estructurado. Un índice de Azure AI Search puede tener, al mismo tiempo: un campo de texto buscable (el chunk), un campo vector (su embedding), y campos filtrables no-vector (organismo, fecha, estado, categoría). Se simplifica todo a **un solo storage**.

```
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from azure.core.credentials import AzureKeyCredential

search_client = SearchClient(endpoint=search_endpoint, index_name="pliegos-chunks", credential=AzureKeyCredential(
    search_key

# Hybrid search: texto + vector + filtro, en una sola consulta
vector_query = VectorizedQuery(vector=query_embedding, k_nearest_neighbors=5, fields="content_vector")
results = search_client.search(
    search_text="plazo de presentación de ofertas",
    vector_queries=[vector_query],
    filter="organismo eq 'Ministerio de Hacienda'",
    select=["chunk_text", "organismo", "categoria"],
)
```

El tier gratuito alcanza para probar, pero vector search en producción necesita el tier **Basic o superior**.

Azure OpenAI

Acceso a modelos de OpenAI (chat y embeddings) dentro del perímetro de seguridad de Azure. Hacen falta **dos modelos desplegados**:

- Un modelo de **chat/completions** — lo usa cada uno de los 7 nodos de extracción para redactar el campo a partir de los chunks recuperados
- Un modelo de **embeddings** (ej. `text-embedding-3-small`) — vectoriza cada chunk antes de indexarlo en Azure AI Search

AUTENTICACIÓN RECOMENDADA

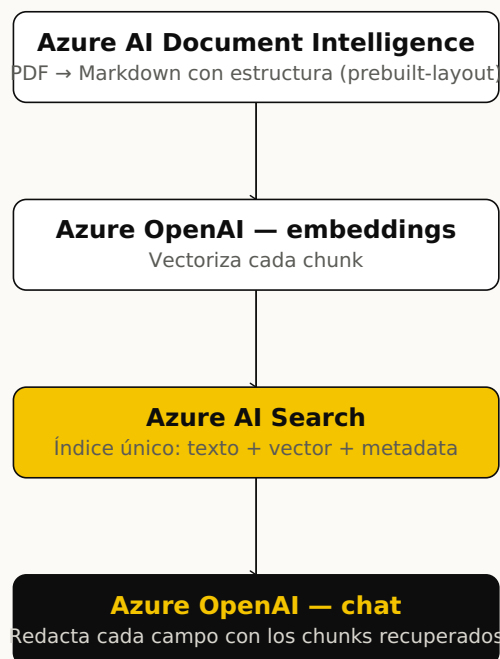
Usar `azure-identity` con `DefaultAzureCredential` (identidad administrada) en vez de API keys sueltas — sobre todo una vez desplegado en Azure Container Apps, donde la identidad administrada evita tener que guardar secretos.

Cómo interactúan entre sí — el flujo completo

De punta a punta, para un pliego nuevo:

1. El PDF sube al backend
2. **Azure AI Document Intelligence** (`prebuilt-layout`) devuelve el documento en Markdown, con estructura preservada
3. Chunking estructural (por sección) + semántico sobre ese Markdown
4. **Azure OpenAI — embeddings** vectoriza cada chunk

5. **Azure AI Search** indexa cada chunk: texto + vector + metadata (organismo, fecha, categoría)
6. Cada uno de los 7 nodos de extracción consulta Azure AI Search con *hybrid search* para traer los chunks candidatos de su categoría
7. **Azure OpenAI — chat** arma el prompt de cada nodo con esos chunks y redacta el campo extraído
8. El nodo de merge y validación arma el JSON final
9. El resultado queda disponible para consultas futuras directamente en Azure AI Search



6 Ejemplo completo, paso a paso

Esta sección es para alguien que sabe programar, LLMs, Python y LangGraph, pero nunca trabajó con un framework de planificación agéntica como BMAD. La idea es sacar la duda de "¿cómo se aplica esto en la práctica?" con un ejemplo real, de punta a punta, sobre este mismo proyecto.

Los "roles" que vas a usar

BMAD invoca distintos *agentes* (Analyst, PM, UX-Designer, Architect, Dev) — son personas/perspectivas distintas que la misma IA adopta según qué skill invocás, no software separado.

Etapas 0 — Planificación (una sola vez, para todo el proyecto)

Se hace en orden, cada paso en un chat nuevo, usando **Claude Opus 4.6**:

1. **bmad-product-brief** — El agente Analyst pregunta qué problema resuelve el proyecto. Se le cuenta: *"Analizar pliegos de licitación en PDF y extraer 7 categorías (objeto y alcance, requisitos de admisibilidad, garantías, plazos clave, criterios de evaluación, causales de rechazo, anexos obligatorios), guardarlas estructuradas y buscables sobre Azure AI Search. El MVP no incluye scoring ni el Radar de oportunidades todavía."* Sigue preguntando sobre usuarios y alcance. Al final: `_bmad-output/planning-artifacts/product-brief.md`.
2. **bmad-prd** — Toma el brief y arma los requisitos funcionales: el pipeline de 7 nodos, y las 3 pantallas de frontend (Dashboard, Análisis de Pliegos, Historial). Sale `prd.md`.
3. **bmad-ux** (agente UX-Designer) — Diseña las 3 pantallas, usando las capturas de inspiración como referencia de estilo. Sale el documento de diseño UX.
4. **bmad-architecture** (agente Architect) — Acá se vuelca la arquitectura de capas simples + adapters livianos, el stack 100% Azure (Document Intelligence, AI Search, OpenAI, LangGraph), y los 7 nodos + merge + storage. Sale `architecture.md`.
5. **bmad-create-epics-and-stories** (agente PM) — Usa PRD + arquitectura + UX juntos. Genera épicas, por ejemplo: *"Épica: Pipeline de extracción"* (9 historias: ingesta, chunking, 7 nodos, merge) y *"Épica: Frontend"* (3 historias: Dashboard, Análisis de Pliegos, Historial).

6. `bmad-check-implementation-readiness` (agente Architect, con Sonnet 4.6 alcanza) — Valida que todo sea coherente antes de programar.

Etapa 1 — Arranca la implementación (una sola vez)

```
bmad-sprint-planning
```

El agente Dev genera `sprint-status.yaml` con las 12 historias de ambas épicas.

Etapa 2 — Ejemplo A: una historia simple

Historia: **"Pantalla Historial — listado de pliegos"** (de la épica Frontend). Autocontenida y de bajo riesgo:

Paso	Skill	Modelo	Qué pasa
1	<code>bmad-create-story</code>	Sonnet 4.6	El Dev agent arma el archivo de la historia: tabla con nombre/organismo/fecha/estado, buscador, acciones ver/duplicar/eliminar
2	<code>bmad-dev-story</code>	GPT-5.3-Codex	Implementa el componente de frontend
3	<code>bmad-code-review</code>	GPT-5.3-Codex	Revisa el código generado

Etapa 3 — Ejemplo B: una historia compleja

Historia: **"Nodo de extracción — objeto y alcance"** (de la épica Pipeline). Mismo ciclo de 3 pasos que el Ejemplo A, pero con más cuidado en el primero — acá es donde se define cómo lidiar con la variabilidad de redacción (sección 8):

Paso	Skill	Modelo	Qué pasa
1	<code>bmad-create-story</code>	Opus 4.6	Describe qué recibe (chunks recuperados de Azure AI Search para esta categoría), qué devuelve (texto + evidencia + confianza), y casos borde: texto mencionado en el índice y en el cuerpo con distinta redacción
2	<code>bmad-dev-story</code>	GPT-5.3-Codex	Implementa el nodo: consulta Azure AI Search con hybrid search, arma el prompt con los chunks recuperados, llama a Azure OpenAI, valida con Pydantic
3	<code>bmad-code-review</code>	GPT-5.3-Codex	Revisa el código generado

La diferencia entre el Ejemplo A y el Ejemplo B no está en el ciclo — es el **mismo ciclo de 3 pasos siempre**. Lo que cambia es cuánto cuidado (y qué modelo) conviene poner en el paso 1, según cuánto riesgo tiene la historia.

Etapa 4 — Cierre de épica

Al terminar todas las historias de una épica (ej. las 9 de Pipeline, o las 3 de Frontend):

```
bmad-retrospective
```

7 Extracción robusta ante variabilidad de redacción

Los pliegos no usan siempre el mismo vocabulario: un mismo concepto (por ejemplo, el plazo de

presentación) puede aparecer como "fecha de apertura de sobres", "plazo de entrega de ofertas" o "fecha límite de recepción" según el organismo. La extracción **no puede depender de matching literal de palabras clave** — tiene que ser semántica.

- 1. Prompt por significado, no por nombre**

El prompt describe qué es el concepto ("fecha límite en que los oferentes deben presentar su propuesta, puede aparecer como...") en vez de pedir la frase exacta.
- 2. Glosario de sinónimos por categoría**

Se arma con pliegos reales a medida que se procesan, y se suma como contexto al prompt de extracción.
- 3. Hybrid search antes de extraer**

Azure AI Search busca los chunks más parecidos al *concepto* de cada campo (vector) combinado con términos exactos (keyword), no solo a su título literal — esos candidatos son el contexto que recibe Azure OpenAI.
- 4. Evidencia + nivel de confianza**

El modelo de chat devuelve además la cita del chunk de origen y un nivel de confianza. Confianza baja o campo no encontrado → se marca para revisión humana, nunca se inventa un valor.

GOLDEN SET

Armar 15-20 pliegos anotados a mano (verdad de referencia) para medir qué tan bien se extrae cada campo frente a redacciones distintas, y ajustar prompt/glosario según los errores reales — no en base a suposiciones.

8 Frontend del MVP

CORRECCIÓN IMPORTANTE

El frontend **todavía no está construido**. Las capturas usadas como referencia eran solo inspiración de estilo y flujo — no una pantalla ya hecha para conectar. Diseñarlo y construirlo es trabajo real dentro del plan, y va a salir como historias propias en `bmad-create-epics-and-stories` (ver sección 4, "Diseño de UX").

Diseño de referencia para las tres pantallas principales (a validar/ajustar en `bmad-ux`):

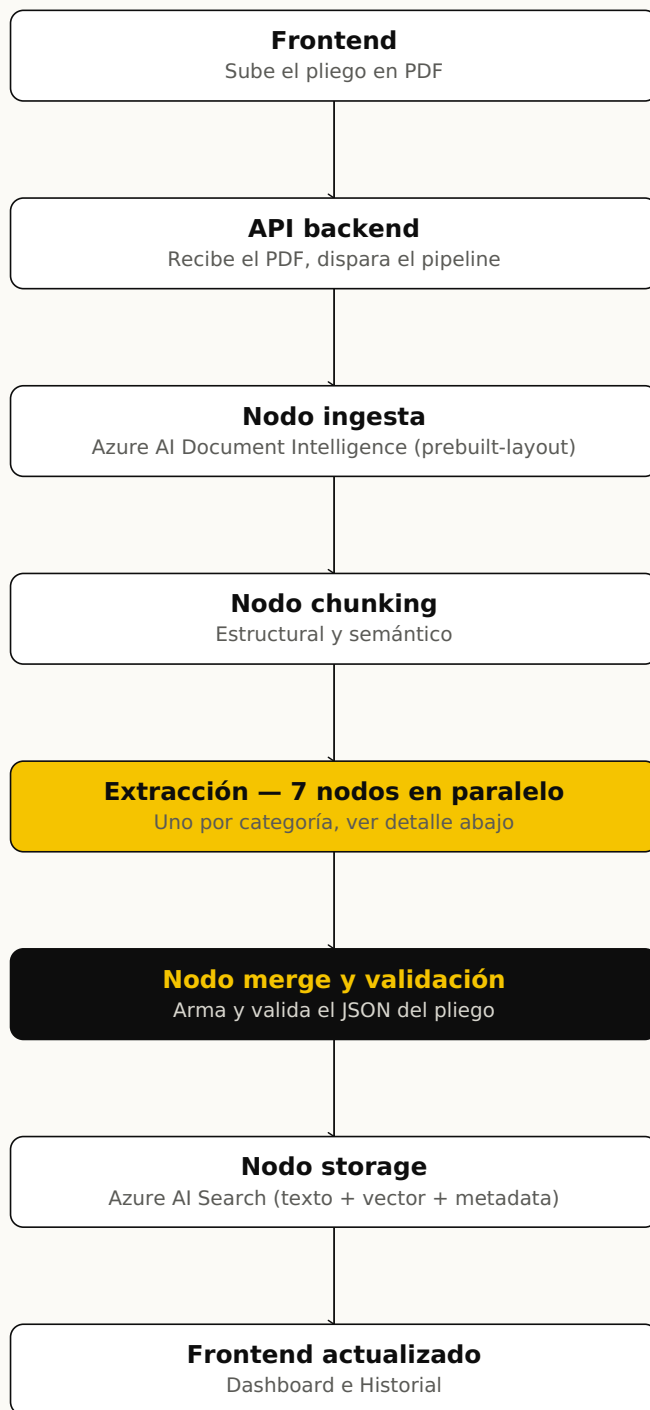
Pantalla	Función
Dashboard	Resumen: pliegos analizados, último análisis, y una tabla de los últimos análisis (pliego, organismo, fecha, estado)
Análisis de Pliegos → Nuevo	Carga del PDF (drag & drop o selector de archivo) y botón "Analizar con IA" que dispara el pipeline
Historial	Listado completo de pliegos analizados, con búsqueda por pliego u organismo, y acciones (ver, duplicar, eliminar)

El sidebar está pensado para escalar desde el diseño: separar **Plataforma** (lo que se construye en este MVP: Dashboard, Análisis de Pliegos, Historial, Configuración) de **Próximamente** (Radar de oportunidades, Chat IA, CRM, Calendario), para que agregar una sección nueva sea sumar un ítem a esa lista, no reestructurar la navegación.

Radar de oportunidades: sección a cargo de otro integrante del equipo, que va a traer oportunidades de licitación a la web. Queda reservada en el sidebar (Próximamente) pero no se construye en este MVP — el foco actual es únicamente el análisis del pliego que se sube manualmente.

9 Arquitectura end-to-end del MVP (nodos)

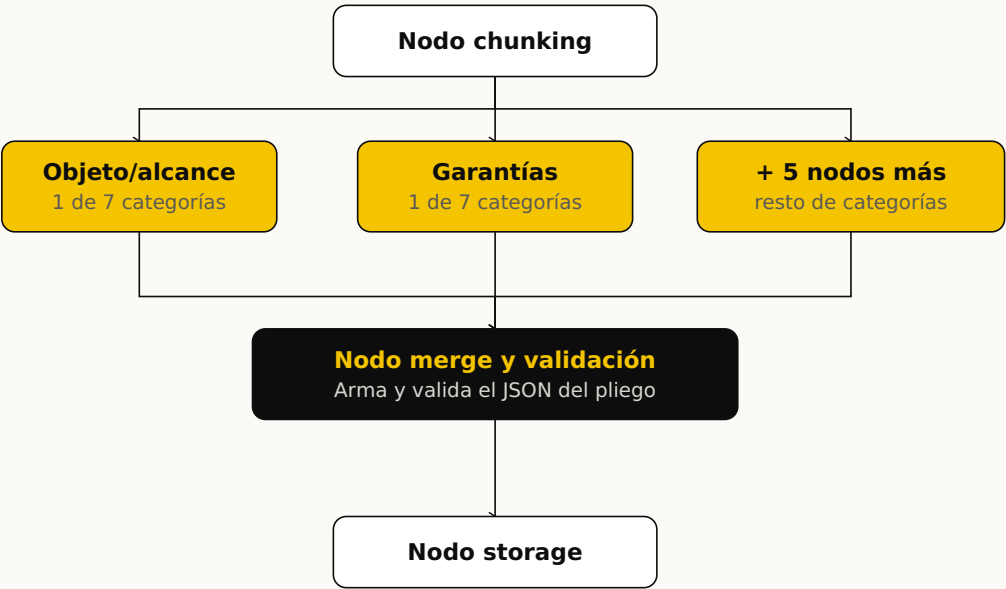
Flujo completo desde que se sube el PDF hasta que el resultado aparece en el frontend. La extracción no es un solo nodo — cada una de las 7 categorías es su propio nodo, corriendo en paralelo, y un nodo de merge y validación arma el JSON final antes de guardarlo:



Detalle: extracción como fan-out de 7 nodos

Cada categoría corre en su propio nodo, con su propio prompt y su propia búsqueda semántica (glosario

de sinónimos de la sección 8). Los 7 convergen en un único nodo de merge y validación:



Los 7 nodos de extracción, uno por categoría:

1. Objeto y alcance
2. Requisitos de admisibilidad
3. Garantías
4. Plazos clave
5. Criterios de evaluación
6. Causales de rechazo
7. Anexos obligatorios

Paso	Componente	Qué hace
1	Frontend	El usuario sube el pliego en PDF desde "Análisis de Pliegos → Nuevo"
2	API backend	Recibe el PDF y dispara el grafo de LangGraph
3	Nodo ingesta	Azure AI Document Intelligence (prebuilt-layout) convierte el PDF a Markdown estructurado
4	Nodo chunking	Chunking estructural (por sección) + semántico
5	Extracción (7 nodos en paralelo)	Un nodo por categoría, cada uno con su propio prompt y búsqueda semántica
6	Nodo merge y validación	Combina los 7 resultados en un solo JSON y lo valida contra el schema Pydantic completo
7	Nodo storage	Indexa en Azure AI Search: texto, vector y metadata (organismo, fecha, estado) en un mismo documento
8	Frontend actualizado	El resultado aparece en Dashboard e Historial

Cada nodo corresponde a un archivo propio en `nodes/` (ver la estructura de carpetas de la sección 12), lo que permite testear cada uno por separado antes de conectarlos, y es la unidad natural para llevar como una historia independiente de BMAD (ej. "Nodo objeto y alcance").

10 Cómo optimizar el consumo de créditos

Con todo en Azure desde el día uno, el gasto se reparte entre varios servicios distintos — no es un solo

medidor. Conviene pensarlo por pieza:

Azure AI Document Intelligence

Cachear el resultado

Se cobra por página analizada. Guardar el Markdown resultante (por hash del PDF) y nunca volver a analizar el mismo pliego dos veces, ni siquiera en pruebas repetidas.

Un solo modelo para todo

`prebuilt-layout` alcanza para nativos y escaneados por igual — no hace falta entrenar ni pagar un modelo custom para el MVP.

Azure AI Search

Elegir bien el tier

Tier gratuito para desarrollo/pruebas; recién pasar a Basic cuando el volumen real de pliegos lo justifique — no antes.

Reusar embeddings

Si un pliego ya fue indexado, no volver a generar embeddings de sus chunks — cada llamada de embeddings a Azure OpenAI también tiene costo.

Azure OpenAI

Achicar el contexto por nodo

Cada uno de los 7 nodos de extracción solo debería recibir los 2-3 chunks que trajo el hybrid search para su categoría — nunca el pliego completo. Es la diferencia entre pagar por 300 palabras o por 30 páginas, en cada una de las 7 llamadas.

Modelo según la tarea

No hace falta el modelo más caro para las 7 llamadas de extracción si un modelo más chico rinde igual de bien en el golden set — probar ambos y medir antes de decidir.

GitHub Copilot (BMAD)

Agrupar respuestas

Cada mensaje premium cuenta como una solicitud contra el cupo, sin importar el largo. Si un agente pregunta 3 cosas, responderlas juntas en un solo mensaje en vez de mandar 3 mensajes sueltos.

Modelo fuerte solo donde importa

Reservar Opus 4.6 para planificación e historias delicadas; Auto para lo administrativo (ver la tabla de modelos en la sección 4).

Cómputo (Azure Container Apps)

SCALE-TO-ZERO

Configurar el plan de consumo de Container Apps con escalado a cero réplicas cuando no hay tráfico — para un MVP con uso esporádico (subidas manuales de pliegos), no tiene sentido pagar por cómputo idle entre pruebas o demos.

Monitoreo

Configurar **Azure Cost Management** con un presupuesto (budget) y alertas por servicio desde el primer día — así se detecta un consumo anormal (ej. un loop reprocesando el mismo pliego) antes de que aparezca en la factura del mes, no después.

11 Plan de entrega inicial — estimación revisada

POR QUÉ CAMBIÓ EL NÚMERO

El plan original de "2 semanas" asumía un solo nodo de extracción y un frontend ya construido para conectar. Con los 7 nodos + merge, el frontend como trabajo real por diseñar y construir, y el setup de recursos de Azure, el estimado sube.

Bloque	Estimado
Planificación BMAD (brief, PRD, UX, arquitectura, épicas/historias, chequeo)	1-3 días
Setup de recursos de Azure (Document Intelligence, AI Search, OpenAI, Container Apps) + entorno + ramas	1-1.5 días
Diseño y construcción del frontend (3 pantallas)	2-3 días
Nodo ingesta (Document Intelligence) + nodo chunking	1.5-2 días
7 nodos de extracción (<code>bmad-create-story</code> → <code>bmad-dev-story</code> → <code>bmad-code-review</code> c/u)	4-5 días
Nodo merge/validación + nodo storage (índice de Azure AI Search)	1-1.5 días
Conectar frontend con la API + pruebas de integración	1 día
Pruebas contra 10-15 pliegos de muestra + ajuste de glosario/prompts	1.5-2.5 días
Total estimado	~13.5-19.5 días hábiles (2.5-4 semanas)

El mayor riesgo para que esto se estire es la extracción sobre pliegos escaneados y el ajuste del glosario de sinónimos contra el golden set — depende de qué tan parejos o dispersos sean los pliegos reales que se usen para probar.

12 Estructura de carpetas — Capas simples + adapters livianos

Se descartó la arquitectura hexagonal completa (puertos formales + 5 capas). Con Azure ya fijo como stack — sin migración de nube pendiente — y con todo el desarrollo llevado historia por historia con BMAD, conviene una versión más liviana: **una clase por servicio externo (adapter), sin interfaz formal de por medio, y un archivo por nodo**.

POR QUÉ ESTA VERSIÓN Y NO LA HEXAGONAL COMPLETA

Con BMAD como única metodología, la arquitectura que más rinde es la que se describe fácil en `bmad-architecture` y mapea 1 a 1 con `bmad-create-epics-and-stories`. "Una clase por servicio de Azure, un archivo por nodo" es una frase y una historia por archivo. Los puertos formales solo pagan su ceremonia cuando hay 2+ implementaciones intercambiables de lo mismo — acá no las va a haber, porque el compromiso con Azure ya está tomado.

Lo que sí se mantiene, porque no cuesta nada y rinde en los tests: cada adapter sigue siendo una clase con métodos claros, así que en los tests se reemplaza por un doble falso sin pegarle a Azure real ni gastar créditos — el beneficio de testeabilidad de puertos y adaptadores, sin la capa de interfaces formales.

Carpeta Qué vive ahí

schemas.py Pydantic: `PliegoExtraido` y las 7 categorías

adapters/ Una clase por servicio externo de Azure (Document Intelligence, AI Search, OpenAI), sin interfaz formal

nodes/ Un archivo por nodo del grafo — la unidad natural de cada historia de BMAD

graph.py Arma el grafo completo de LangGraph

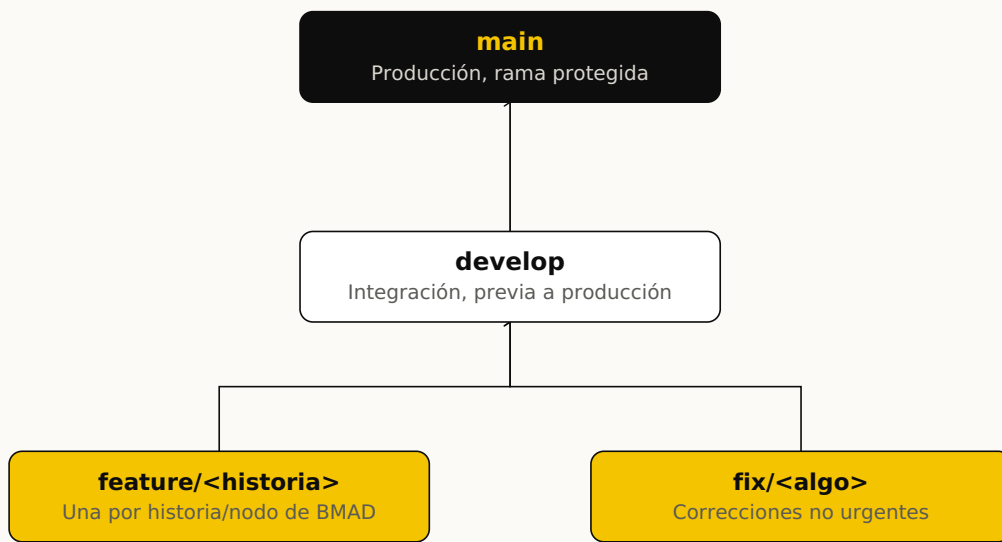
api/ Endpoints que expone el backend al frontend

```
src/
├── schemas.py                # Pydantic: PliegoExtraido + 7 categorías
├── adapters/                 # una clase por servicio externo, sin interfaz formal
│   ├── document_intelligence.py # wrapper de Azure Document Intelligence
│   ├── ai_search.py           # wrapper de Azure AI Search
│   └── openai_client.py       # wrapper de Azure OpenAI (chat + embeddings)
├── nodes/                   # un archivo por nodo del grafo
│   ├── ingesta.py
│   ├── chunking.py
│   ├── extraccion/
│   │   ├── objeto_alcance.py
│   │   ├── requisitos_admisibilidad.py
│   │   ├── garantias.py
│   │   ├── plazos_clave.py
│   │   ├── criterios_evaluacion.py
│   │   ├── causales_rechazo.py
│   │   └── anexos_obligatorios.py
│   ├── merge_validacion.py
│   └── storage.py
├── graph.py                 # arma el grafo de LangGraph
├── api/
│   └── routes.py            # endpoints para el frontend
├── _bmad/                   # instalado por BMAD Method
├── docs/                    # brief.md, prd.md, architecture.md (BMAD)
├── data/
│   ├── raw/
│   └── processed/
└── tests/
```

Cada clase de `adapters/` expone métodos claros (`analyze(pdf) -> Markdown` , `search(query) -> chunks` , `embed(text) -> vector`), así que en los tests se reemplaza por un doble falso que devuelve datos fijos, sin necesidad de tener los servicios de Azure corriendo ni gastar una sola llamada real durante los tests.

13 Estrategia de ramas (Git)

Una versión simplificada de Git Flow, pensada para un equipo chico: dos ramas permanentes (`main` y `develop`) y ramas temporales por historia/fix que se borran después de mergear.



Configuración inicial (una sola vez)

```
git checkout -b develop
git push -u origin develop
```

A partir de acá, `main` solo recibe merges desde `develop` — nunca se trabaja directo sobre `main`.

Por cada historia (feature)

Conviene nombrar la rama igual que la historia que salió de `bmad-create-epics-and-stories`, para que quede trazable:

```
git checkout develop
git pull
git checkout -b feature/nodo-objeto-alcance

# ... trabajar, correr bmad-dev-story, comitear ...

git push -u origin feature/nodo-objeto-alcance
# abrir Pull Request hacia develop
```

Para un fix (bug no urgente)

Misma dinámica que una feature, pero con el prefijo `fix/`:

```
git checkout develop
git pull
git checkout -b fix/chunking-limite-tokens

# ... arreglar, comitear ...

git push -u origin fix/chunking-limite-tokens
# abrir Pull Request hacia develop
```

Cuando develop está lista para entregar

```
# Pull Request de develop hacia main
# después de mergear, opcional: taggear la versión
git checkout main
git pull
git tag v0.1.0
git push --tags
```

CONVENCIÓN DE NOMBRES

`feature/` para historias nuevas, `fix/` para correcciones no urgentes sobre código ya en `develop`. Usar minúsculas y guiones (kebab-case), sin espacios: `feature/nodo-garantias`, no `feature/Nodo Garantias`.

CASO URGENTE EN PRODUCCIÓN (OPCIONAL, PARA MÁS ADELANTE)

Si aparece un bug crítico ya en producción (`main`) y no puede esperar a pasar por `develop`, se usa una rama `hotfix/<algo>` que sale de `main` y se mergea de vuelta tanto a `main` como a `develop`. No hace falta para el MVP, pero es la práctica estándar cuando el sistema ya esté en uso real.

14 Próximos pasos inmediatos

- 1 Instalar Git, Node.js 20.12+, Python 3.11+, `uv` y Azure CLI en el entorno del equipo.
- 2 Crear los recursos de Azure: Document Intelligence, AI Search y OpenAI (con un modelo de chat y uno de embeddings desplegados).
- 3 Correr `npx bmad-method install`, elegir el módulo "BMad Method" y GitHub Copilot como integración.
- 4 Invocar `bmad-help`, después `bmad-product-brief` y `bmad-prd` (usando el borrador ya armado para el brief).
- 5 Correr `bmad-ux`, `bmad-architecture` y `bmad-create-epics-and-stories`, en ese orden.
- 6 Armar la muestra de 10-15 pliegos reales (mezcla nativos/escaneados) y arrancar la implementación historia por historia.